Rastreamento e Captura do Veículo

1. Introdução

Este módulo é responsável pela **detecção, rastreamento e captura automática de veículos** em fluxo de vídeo, servindo como etapa intermediária entre:

- a aquisição de imagens (câmeras/stream),
- e o processamento posterior (detecção de placas + OCR).

A solução utiliza o modelo YOLO para detecção e o algoritmo **ByteTrack** para rastreamento persistente, permitindo identificar veículos únicos ao longo dos frames e evitar capturas duplicadas.

2. Arquitetura Utilizada

YOLOv8 (Detecção)

O modelo `yolov8n.pt` foi utilizado para identificar objetos do tipo veículo.

Justificativa:

- leve e eficiente para processamento em tempo real;
- adequado para execução em CPU;
- boa generalização com dataset COCO.

ByteTrack (Rastreamento)

O rastreamento é realizado via:

```
tracker="bytetrack.yaml"
```

Função:

- atribuir um **ID único** para cada veículo detectado;
- manter esse ID ao longo dos frames;
- permitir controle de captura (evitar múltiplas fotos do mesmo carro).

3. Fonte de Vídeo

O sistema suporta múltiplas entradas:

- Webcam (`0`)
- Arquivos locais (`.mp4`)
- Streams IP (`RTSP/HTTP`)

No experimento, foi utilizada uma câmera pública:

```
video_source="https://appnittrans.niteroi.rj.gov.br:8888/000024/last_video.mp4"
```

Justificativa:

- simula cenário real de monitoramento urbano;
- permite validar o sistema em condições reais (tráfego, iluminação, oclusões).

4. Filtragem por Classe

```
CAR_CLASS=2
```

Classe **2** corresponde a **carro** no dataset COCO.

Motivação:

- reduzir falsos positivos;
- evitar processamento de objetos irrelevantes (pessoas, motos, etc.);
- focar exclusivamente no problema de placas veiculares.

5. Zona de Interesse (ROI)

Foi definida uma região específica do frame:

```
ROI_X_MIN=0.10
ROI_X_MAX=0.90
ROI_Y_MIN=0.55
ROI_Y_MAX=0.70
```

Objetivo:

- capturar veículos apenas quando estão em posição ideal;
- evitar:
  - carros muito distantes;
  - veículos parcialmente visíveis;
  - detecções irrelevantes nas bordas.

Benefícios:

- melhora a qualidade das imagens capturadas;
- aumenta a taxa de sucesso do OCR posteriormente;
- reduz processamento desnecessário.

6. Filtro de Qualidade da Detecção

```
MIN_BOX_WIDTH=120
MIN_BOX_HEIGHT=70
```

Função:

- garantir que apenas veículos com tamanho mínimo sejam processados.

Justificativa:

- veículos pequenos → placas ilegíveis;

- evita salvar imagens sem valor para o sistema.

7. Rastreamento com ID Persistente

O YOLO com `persist=True` mantém IDs consistentes:

```
track_id=int(box.id[0])
```

Uso prático:

- cada carro recebe um ID único;
- permite saber se aquele veículo já foi processado.

8. Controle de Captura (Evitar Duplicidade)

```
saved_track_ids=set()
```

Lógica:

- salva imagem apenas na **primeira vez que o veículo aparece na ROI**;
- ignora frames seguintes do mesmo carro.

Benefícios:

- evita redundância;
- economiza armazenamento;
- simplifica pipeline downstream (OCR).

9. Recorte Inteligente do Veículo

Antes de salvar, é aplicado um recorte com margem:

```
CROP_MARGIN_X=0.12
CROP_MARGIN_Y=0.12
```

Motivo:

- incluir contexto ao redor do carro;
- evitar cortes muito justos que possam perder a placa.

Resultado:

- imagens mais completas e úteis para o processamento posterior.

10. Critérios para Captura da Imagem

Um veículo só é salvo se:

✔️ possui `track_id` válido

✔️ ainda não foi salvo anteriormente

✔️ está dentro da ROI

✔️ atende ao tamanho mínimo

Isso garante **alta qualidade e relevância dos dados coletados**.

11. Armazenamento das Imagens

As imagens são salvas automaticamente:

```
output_dir="fotos_carros"
```

Nome dos arquivos:

```
carro_id{track_id}_{timestamp}_{contador}.png
```

Características:

- timestamp único → evita sobrescrita;
- associação direta com o ID do veículo;
- compressão desativada → preserva qualidade máxima.

12. Visualização em Tempo Real

O sistema permite exibir:

- bounding boxes
- ID do veículo
- confiança da detecção
- zona útil (ROI)

```
show_stream=True
```

Finalidade:

- depuração;
- validação visual do sistema;
- ajuste fino dos parâmetros.

13. Fluxo Completo do Sistema

1. Captura frame do vídeo
2. Executa detecção com YOLO
3. Aplica rastreamento com ByteTrack
4. Filtra por classe (carro)
5. Verifica ROI e tamanho
6. Associa ID ao veículo
7. Salva imagem (uma única vez por ID)
8. Exibe resultado (opcional)

14. Justificativa da Abordagem

A utilização de detecção + rastreamento foi essencial porque:

- apenas detecção geraria múltiplas capturas do mesmo carro;

- o rastreamento permite identificar veículos únicos;
- melhora eficiência do sistema como um todo;
- prepara dados de melhor qualidade para o OCR.

Além disso:

- o uso de ROI e filtros reduz ruído;
- a captura seletiva melhora a precisão do pipeline completo;
- o sistema se torna aplicável em cenários reais de vigilância.

15. Conclusão

O módulo de rastreamento desenvolvido garante uma **coleta inteligente, eficiente e não redundante de veículos**, sendo fundamental para alimentar o sistema de detecção de placas.

A integração entre YOLO e ByteTrack, aliada a filtros espaciais e dimensionais, resulta em um pipeline robusto e adequado para aplicações reais de monitoramento e identificação veicular.